image, which is why the command took a little while to run. Rather than download the image in one go, it downloaded the parts—known as *layers*—that make up the image. So Docker provides a seamless mechanism for delivering exactly the images we need, when we need them.

Why So Slow?

If you run the previous command yourself, there's one slight flaw you may notice: it takes a *looonnng* time. I know interpreted languages like Ruby are slow, but this is ridiculous.

The prior discussion helps to explain why the command took so long. The thing that took the time wasn't executing our tiny Ruby script; it was downloading the `ruby:2.6` image over the network. Whenever you start a container based on an image that you haven't used before, Docker will need to download it first.

Although images are typically *much* smaller than VMs—MBs rather than GBs—waiting 20 seconds for every Docker command would be pretty frustrating. Thankfully, we don't have to. Docker stores downloaded images locally, so the next time you start a container based on the same image, it starts up at virtually native speed. Docker even caches *individual layers* of the image, allowing for reuse of layers between images, as we'll see shortly.

Let's see this for ourselves. Try running the same command for a second time.

```
$ docker run ruby:2.6 ruby -e "puts :hello"
hello
```

Wow. Much faster this time—no output about images being downloaded.

Cleaning Up After Ourselves

Each time we run the `docker run` command, Docker creates a *new* container to run the command. We've now run our Ruby script twice, so we have two virtually identical containers for running this Ruby script.

To list the running containers, we use:

```
$ docker ps
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS    NAMES
```

As you can see, there are no running containers—that's because when our Ruby command terminated, the container running it also terminated. However, unless we tell it otherwise, Docker will keep this stopped container around in case we want to use it again.